

mcintegrate.h

VecDoub torusmap(const VecDoub &s) {

Return the mapping from s to z defined by the last equation in (7.7.7), mapping the other coordinates by the identity map.

```
    VecDoub xx(s);
    xx[2] = 0.2*log(5.*s[2]);
    return xx;
}
```

Code for the actual integration now looks like this:

```
VecDoub slo(3), shi(3);
slo[0] = 1.; shi[0] = 4.;
slo[1] = -3.; shi[1] = 4.;
slo[2] = 0.2*exp(5.*(-1.)); shi[2] = 0.2*exp(5.*(1.));
MCIntegrate mymc2(slo,shi,torusfuncs,torusregion,torusmap,10201);
mymc2.step(1000000);
mymc2.calcanswers();
```

If you think for a minute, you will realize that equation (7.7.7) was useful only because the part of the integrand that we wanted to eliminate (e^{5z}) was both integrable analytically and had an integral that could be analytically inverted. (Compare §7.3.2.) In general these properties will not hold. Question: What then? Answer: Pull out of the integrand the “best” factor that *can* be integrated and inverted. The criterion for “best” is to try to reduce the remaining integrand to a function that is as close as possible to constant.

The limiting case is instructive: If you manage to make the integrand f *exactly* constant, and if the region V , of known volume, *exactly* encloses the desired region W , then the average of f that you compute will be exactly its constant value, and the error estimate in equation (7.7.1) will exactly vanish. You will, in fact, have done the integral exactly, and the Monte Carlo numerical evaluations are superfluous. So, backing off from the extreme limiting case, *to the extent* that you are able to make f approximately constant by change of variable, and *to the extent* that you can sample a region only slightly larger than W , you will increase the accuracy of the Monte Carlo integral. This technique is generically called *reduction of variance* in the literature.

The fundamental disadvantage of simple Monte Carlo integration is that its accuracy increases only as the square root of N , the number of sampled points. If your accuracy requirements are modest, or if your computer is large, then the technique is highly recommended as one of great generality. In §7.8 and §7.9 we will see that there are techniques available for “breaking the square root of N barrier” and achieving, at least in some cases, higher accuracy with fewer function evaluations.

There should be nothing surprising in the implementation of MCIntegrate. The constructor stores pointers to the user functions, makes an otherwise superfluous call to funcs just to find out the size of returned vector, and then sizes the sum and answer vectors accordingly. The step and calcanswer methods implement exactly equations (7.7.1) and (7.7.2).

mcintegrate.h

```
MCIntegrate::MCIntegrate(const VecDoub &xlow, const VecDoub &xhigh,
    VecDoub funcs(const VecDoub &), Bool inregion(const VecDoub &),
    VecDoub xmap(const VecDoub &), Int ranseed)
: ndim(xlow.size()), n(0), xlo(xlow), xhi(xhigh), x(ndim), xx(ndim),
  funcsp(funcs), xmap(xmap), inregionp(inregion), vol(1.), ran(ranseed) {
    if (xmap) nfun = funcs(xmap(xlo)).size();
    else nfun = funcs(xlo).size();
    ff.resize(nfun);
    fferr.resize(nfun);
```

```

    fn.resize(nfun);
    sf.assign(nfun,0.);
    sferr.assign(nfun,0.);
    for (Int j=0;j<ndim;j++) vol *= abs(xhi[j]-xlo[j]);
}

void MCIntegrate::step(Int nstep) {
    Int i,j;
    for (i=0;i<nstep;i++) {
        for (j=0;j<ndim;j++)
            x[j] = xlo[j]+(xhi[j]-xlo[j])*ran.doub();
        if (xmapp) xx = (*xmapp)(x);
        else xx = x;
        if ((*inregionp)(xx)) {
            fn = (*funcsp)(xx);
            for (j=0;j<nfun;j++) {
                sf[j] += fn[j];
                sferr[j] += SQR(fn[j]);
            }
        }
    }
    n += nstep;
}

void MCIntegrate::calcanswers(){
    for (Int j=0;j<nfun;j++) {
        ff[j] = vol*sf[j]/n;
        fferr[j] = vol*sqrt((sferr[j]/n-SQR(sf[j]/n))/n);
    }
}

```

CITED REFERENCES AND FURTHER READING:

- Robert, C.P., and Casella, G. 2006, *Monte Carlo Statistical Methods*, 2nd ed. (New York: Springer)
- Sobol', I.M. 1994, *A Primer for the Monte Carlo Method* (Boca Raton, FL: CRC Press).
- Hammersley, J.M., and Handscomb, D.C. 1964, *Monte Carlo Methods* (London: Methuen).
- Gentle, J.E. 2003, *Random Number Generation and Monte Carlo Methods*, 2nd ed. (New York: Springer), Chapter 7.
- Shreider, Yu. A. (ed.) 1966, *The Monte Carlo Method* (Oxford: Pergamon).
- Kalos, M.H., and Whitlock, P.A. 1986, *Monte Carlo Methods: Volume 1: Basics* (New York: Wiley).

7.8 Quasi- (that is, Sub-) Random Sequences

We have just seen that choosing N points uniformly randomly in an n -dimensional space leads to an error term in Monte Carlo integration that decreases as $1/\sqrt{N}$. In essence, each new point sampled adds linearly to an accumulated sum that will become the function average, and also linearly to an accumulated sum of squares that will become the variance (equation 7.7.2). The estimated error comes from the square root of this variance, hence the power $N^{-1/2}$.

Just because this square-root convergence is familiar does not, however, mean that it is inevitable. A simple counterexample is to choose sample points that lie on a Cartesian grid, and to sample each grid point exactly once (in whatever order).